# GRIDDY

Project documentation

griddy-survey.vercel.app · github.com/suhasp3/GRIDDY · October 2026

Contents on the next page ↓

# Contents


## 1. Overview

What GRIDDY is and how it fits together

## 2. The codebase

Pages, data model, state, key files

## 3. Building locally

Setup, commands, before you push

## 4. Deploying to Vercel

Push to deploy, the feature workflow, rollbacks

## 5. Keeping it healthy

vercel.json, checks, and recent cleanup

# 1. Overview

↑ Contents

GRIDDY is a browser-only tool for building grid survey questions and exporting them to Qualtrics. There's no backend or database.

- Researchers design a grid in the editor and preview it live.
- Surveys are saved in the browser's `localStorage`. They can be backed up as JSON.
- Export produces a `.qsf` file that Qualtrics imports directly, or a JS snippet to paste.
- Respondents answer inside Qualtrics. Answers are stored as JSON in embedded-data fields (`GridAssignments`, or `GridPrefills` / `GridResponses` in experimental mode).

```
Editor (ConfigPanel) → EditorContext (state) → PreviewPanel
                                ↓
          surveysApi → localStorage    qualtricsExport → .qsf / JS → Qualtrics
```

**Stack:** React 18, TypeScript, Vite, Tailwind, React Router, Vitest. Hosted on Vercel as static files.

# 2. The codebase

↑ Contents

## Pages

| URL | File | Purpose |
|---|---|---|
| `/` | `App.tsx` | The editor and live preview |
| `/history` | `pages/HistoryPage.tsx` | My Surveys: open, delete, JSON backup |
| `/history/combine` | `pages/CombineSurveysPage.tsx` | Combine several surveys into one `.qsf` |

## Data model (`src/grid-types.ts`)

A survey is a `GridConfig` with four parts: **layout** (rows, cols, center cell, barriers), **tuning** (sizes), **survey** (categories, selection mode) and optional **experimental** (pre-filled grid + response labels). Cells are keyed `"r{row}-c{col}"`, 1-based.

## State (`src/EditorContext.tsx`)

One reducer holds the survey being edited. Components read it with `useEditor()` and change it by dispatching actions such as `updateLayout`, `updateSurvey`, `setConfig` or `newSurvey`.

## Key files

| | |
|---|---|
| `components/ConfigPanel.tsx` | Left-hand editor UI |
| `components/PreviewPanel.tsx` | Live preview and Export modal |
| `components/RespondentPreview.tsx` | Runs the real exported script |
| `lib/qualtricsExport.ts` | Builds the Qualtrics JS and `.qsf` |
| `lib/surveysApi.ts` | Save, load and import surveys (localStorage) |
| `lib/imageInput.ts` | Compresses uploaded images into the survey |

## Adding a new setting

Add it to the type → give it a default in `createDefaultConfig` and `normalizeConfig` → add a control in `ConfigPanel` → render it in `PreviewPanel` **and** in `qualtricsExport.ts` → add a test.

> The Qualtrics script is plain JavaScript, not React, so visual features have to be built in both places. Use the respondent preview to check that they match.

# 3. Building locally

↑ Contents

Requires Node 20+ and git. No environment variables are needed.

```
git clone https://github.com/suhasp3/GRIDDY.git
cd GRIDDY
npm install
npm run dev        # http://localhost:5173
```

| Command | Does |
|---|---|
| `npm run dev` | Dev server with hot reload |
| `npm run build` | Type-check, then bundle to `dist/`. This is what Vercel runs. |
| `npm run preview` | Serve the built `dist/` locally |
| `npm test` | Run the Vitest tests |
| `npm run lint` | ESLint |

> **Before pushing:** run `npm run build` (and ideally `npm test` and `npm run lint`). If it fails locally, the Vercel deploy fails too. Unused variables count as build errors.

Saved surveys are stored per site: localhost and the live site keep separate lists. Use Export/Import JSON to move them.

# 4. Deploying to Vercel

↑ Contents

Vercel is connected to the GitHub repo. **Pushing is the deploy.** There's no separate deploy command.

| Push to… | Result |
|---|---|
| `main` | Live site updates in about a minute |
| Any other branch or PR | A separate preview URL. The live site is untouched. |

## Adding a feature

```
git switch main && git pull
git switch -c feature/my-change
# ...build it, check npm run build...
git add -A && git commit -m "My change"
git push -u origin feature/my-change
gh pr create --base main --fill
```

1. Test the preview URL that Vercel posts on the PR.
2. Wait for the green **Vercel** check. Red means the build failed: click Details.
3. Merge the PR. Production updates automatically.

## If production breaks

- **Fastest:** in the Vercel dashboard, go to Deployments, open the last good one, then ⋯ → *Promote to Production*.
- **Via git:** `git revert <commit>` on main and push.

The Vercel project lives under **suhasp3s-projects**. Ask Suhas for dashboard access.

# 5. Keeping it healthy

↑ Contents

## vercel.json

The app has client-side routes, so there's no real `/history` file. `vercel.json` serves `index.html` for any path that isn't a real file, so loading or refreshing any page works:

```
{ "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }
```

Keep it when you add new routes.

## Checks

`npm run build`, `npm test` and `npm run lint` all pass cleanly on `main`. If one fails, something you changed caused it.

## Cleanup done (October 2026)

- Fixed the 404 when refreshing `/history` and `/history/combine`.
- Updated an outdated test and fixed all lint errors and warnings.
- Removed the unused `LayoutTab.tsx` and `SurveyTab.tsx`, and the unused `localforage` and `react-webcam` packages.